

DevOps Working Knowledge Guide

For Developers — Teammate Audit Application

Docker | Jenkins | Kubernetes | AWS | CI/CD Pipeline

Beginner to Interview-Ready

Chapter 0 — The Big Picture: How It All Connects

Before diving into individual tools, it helps to understand why each one exists and how they talk to each other. Here is the lifecycle of your code — from your laptop to production — in the Teammate project.

The Journey of Your Code

Stage	Who / What Does It	Tool Used
1. You write code	Developer (you)	IDE + Git
2. Push to branch	Developer	Git / GitHub
3. Merge triggers pipeline	Automatic	Jenkins
4. Build & package app	Jenkins runs Maven	Maven / Jenkins
5. Run tests & quality checks	Jenkins	JUnit / SonarQube
6. Build Docker image	Jenkins	Docker
7. Push image to registry	Jenkins	AWS ECR
8. Deploy to Kubernetes	Jenkins / kubectl	Kubernetes on AWS EKS
9. Kubernetes runs containers	K8s cluster	Docker containers in Pods
10. Users access the app	Browser / API client	Load Balancer URL

 **Note:** In your Teammate project: the monolith + extracted microservices (reporting, document, notification, background-jobs) all follow this same pipeline separately.

Visual Architecture: Two EC2 VMs and the Pipeline

VM-1 (Tomcat / App Server) ← VM-2 (Jenkins Server) ← GitHub Repo

Jenkins lives on its own VM so heavy build tasks do not slow down the app server. This is a production best-practice.

Flow step-by-step:

- Developer pushes code to GitHub (e.g., to the 'dev' branch)
- Jenkins detects the change (webhook or polling)
- Jenkins pulls the code, runs mvn clean package, creates a .war/.jar
- Jenkins builds a Docker image from the Dockerfile in the repo
- Jenkins pushes the image to AWS ECR (container registry)
- Jenkins instructs Kubernetes (via kubectl) to roll out the new image
- Kubernetes pulls the image from ECR, creates new Pods, kills old Pods (zero-downtime)
- Users hit the Load Balancer URL and are routed to healthy Pods

 **Interview Tip:** Always say: 'Jenkins is our CI/CD automation server. It sits on its own EC2 VM (VM-2) separate from the application server (VM-1) to avoid resource contention.'

Chapter 1 — AWS: Where Everything Runs

AWS (Amazon Web Services) is the cloud platform where your entire infrastructure lives. You do not buy physical servers — you rent virtual machines, databases, and storage from AWS and pay only for what you use.

1.1 Core AWS Services You Use

Service	Full Name	What It Does in Teammate
EC2	Elastic Compute Cloud	Virtual Linux servers — Jenkins VM, Tomcat VM
ECR	Elastic Container Registry	Stores your Docker images (like Docker Hub but private)
EKS	Elastic Kubernetes Service	Managed Kubernetes cluster — runs your containers
RDS	Relational Database Service	Managed MySQL/PostgreSQL database
S3	Simple Storage Service	Stores uploaded audit documents / files
Lambda	AWS Lambda	Serverless functions for event-driven background tasks
IAM	Identity & Access Management	Users, roles, and permissions for AWS resources
VPC	Virtual Private Cloud	Isolated private network for your resources
Security Group	Firewall rules	Controls which ports/IPs can reach your VMs
ELB / ALB	Elastic / Application Load Balancer	Distributes traffic across your K8s pods

1.2 EC2 — Virtual Machines

EC2 gives you a Linux virtual machine in the cloud. You SSH into it just like a physical server.

Key Concepts as a Developer

- **Instance Type:** t2.micro (free tier), t2.medium (for K8s worker nodes). More CPU/RAM = higher cost.
- **AMI:** Amazon Machine Image — the OS template (Amazon Linux, Ubuntu, etc.).
- **Key Pair:** A .pem file for SSH access. DevOps creates this; you use it via MobaXterm or terminal.
- **Security Group:** Acts as a firewall. You **MUST** open specific ports for your app to be accessible. e.g., 8080 for Tomcat, 9090 for Docker containers.
- **Elastic IP:** A fixed public IP (paid). Without it, the IP changes every restart.

Common EC2 Commands You Should Know

```
# Connect to EC2 from terminal
ssh -i mykey.pem ec2-user@<public-ip>

# Check running processes
ps -ef | grep java

# Check port usage
netstat -tlnp | grep 8080

# View application logs
tail -f /path/to/app.log
```

1.3 IAM — Who Can Do What

IAM controls access to AWS resources. DevOps creates IAM users and roles. As a developer, you need to understand:

- **Root Account:** The master account. Never used for daily work. MFA (Google Authenticator) is mandatory.
- **IAM User:** Created for each team member. Limited permissions based on role.
- **IAM Role:** Attached to EC2 instances or EKS worker nodes. Allows them to pull from ECR, write to S3, etc.
- **Access Keys:** Used by Jenkins to authenticate with AWS programmatically (push images to ECR, deploy to EKS).

 **Note:** DevOps sets up IAM. You should know that Jenkins needs an IAM role with ECR push permissions and EKS deployment permissions to run the pipeline.

1.4 S3 — File Storage

In Teammate, auditors upload PDF documents. These files are stored in S3.

- **Bucket:** A container for files (like a folder). e.g., teammate-audit-documents.
- **Object:** Each file stored in S3 is an object. Has a URL like:
`https://s3.amazonaws.com/bucket/file.pdf`
- **Your code (Spring Boot):** Uses AWS SDK to upload/download from S3. The bucket name and region come from `application.properties` or environment variables.

```
# In application.properties (environment-specific)
aws.s3.bucket=teammate-audit-documents-dev
aws.s3.region=ap-south-1
```

 **Interview Tip:** Say: 'In our project, audit document uploads go to S3. The S3 bucket name is environment-specific — dev bucket for dev env, prod bucket for prod env. Jenkins injects the correct bucket name via environment variables during deployment.'

Chapter 2 — Docker: Packaging Your Application

Docker solves the classic problem: 'It works on my machine but not on the server.' Docker packages your application + all its dependencies (Java version, libraries, configs) into a single portable unit called a Container.

2.1 Core Docker Concepts

Concept	Simple Explanation	Analogy
Dockerfile	Recipe/instructions to build the image	Recipe for a dish
Docker Image	Snapshot of your app + dependencies (read-only)	Frozen recipe result
Docker Container	A running instance of an image	The actual dish served
Docker Hub / ECR	Registry to store and share images	Recipe book / warehouse
Docker Compose	Run multi-container apps with one command	Ordering a full meal combo

2.2 Dockerfile — The Most Important File

The Dockerfile lives in your project's root folder (alongside pom.xml). Jenkins reads this file to build the Docker image.

Dockerfile for Your Spring Boot Microservice

File location: /project-root/Dockerfile

```
# Step 1: Start from an official Java base image
FROM openjdk:17-jdk-slim

# Step 2: Who maintains this image (optional but good practice)
LABEL maintainer="yourteam@company.com"

# Step 3: Set a working directory inside the container
WORKDIR /app

# Step 4: Copy the built JAR from Maven into the container
COPY target/teammate-service.jar app.jar

# Step 5: Tell Docker which port the app runs on (documentation only)
EXPOSE 8080

# Step 6: Command to run when container starts
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfile Keywords Explained

Keyword	What It Does	When to Change It
FROM	Base image (Java version). Must match your project's Java.	Change Java version e.g. openjdk:21-jdk-slim
WORKDIR	Working directory inside container	Rarely — /app is standard
COPY	Copies built JAR into image. Path must match Maven output.	If JAR name changes or module path changes
EXPOSE	Documents which port the app listens on	If your server.port changes
ENTRYPOINT	Command to start the app	If you need JVM flags e.g. -Xmx512m
ENV	Set environment variables inside container	For DB URLs, S3 bucket names per environment
RUN	Execute commands during image BUILD (install tools)	If you need OS packages

Passing Environment Config (Very Important)

Your app's application.properties has different values per environment. You do NOT bake env-specific config into the Docker image. Instead, you pass them at runtime:

```
# Option 1: Pass as env vars when running locally
docker run -e SPRING_DATASOURCE_URL=jdbc:mysql://dev-db/teammate \
-e AWS_S3_BUCKET=teammate-dev \
-p 8080:8080 teammate-service:latest

# Option 2: Kubernetes injects them via ConfigMap / Secrets (production)
# Jenkins sets them per environment during deployment
```

 **Note:** One Docker image is built ONCE and promoted across environments (dev → qa → uat → prod). Environment-specific values (DB URL, S3 bucket) are injected by Kubernetes, not baked into the image.

2.3 Essential Docker Commands

Command	What It Does	When You Use It
docker build -t myapp:v1 .	Build image from Dockerfile in current folder	After code change, during pipeline
docker images	List all images on your machine	Verify image was built
docker run -p 8080:8080 myapp:v1	Run container, map host:container port	Local testing

Command	What It Does	When You Use It
<code>docker run -d -p 8080:8080 myapp:v1</code>	Run in detached (background) mode	Keep terminal free
<code>docker ps</code>	List running containers	Check what's running
<code>docker ps -a</code>	List all containers (incl. stopped)	Debug failed containers
<code>docker logs <container-id></code>	View container stdout/stderr logs	Debug app startup errors
<code>docker stop <container-id></code>	Stop a running container	Cleanup
<code>docker rm <container-id></code>	Remove stopped container	Cleanup
<code>docker rmi <image-id></code>	Remove an image	Free disk space
<code>docker exec -it <id> bash</code>	Open a shell inside running container	Debug inside container
<code>docker push ecr-url/myapp:v1</code>	Push image to ECR registry	Jenkins does this
<code>docker pull ecr-url/myapp:v1</code>	Pull image from ECR registry	Kubernetes does this

2.4 Port Mapping Explained

`-p 8080:8080` means: `HOST_PORT : CONTAINER_PORT`

- Left side (8080): port on the EC2 VM / your laptop that you access
- Right side (8080): port the app runs on INSIDE the container
- Example: `-p 9090:8080` means access on port 9090 but app runs on 8080 inside

 **Note:** You must open the HOST port in the EC2 Security Group (inbound rules) for external access.

2.5 Docker in Your Project — What You Actually Do

As a developer on Teammate, here is your involvement:

- **You write the Dockerfile** and keep it updated when Java version changes or JAR name changes.
- **You do NOT manually run `docker build` in production.** Jenkins does that automatically.
- **Locally:** You run your Spring Boot app directly (`java -jar` or from IDE), not via Docker. Docker is for the pipeline.
- **If Jenkins pipeline fails at 'docker build',** you check the Dockerfile and the target/ folder to ensure the JAR was created correctly.

 **Interview Tip:** Say: 'I maintain the Dockerfile for each service. It uses `openjdk:17-jdk-slim` as the base, copies the Maven-built JAR, and exposes port 8080. Environment-specific configuration is not baked in — Kubernetes injects it via ConfigMaps and Secrets.'

Chapter 3 — Jenkins: Automating Build & Deployment

Jenkins is the automation engine that ties everything together. It watches your GitHub branches, and when code changes, it automatically builds, tests, packages, dockerizes, and deploys your application.

3.1 What Jenkins Does (Step by Step)

Jenkins Step	What Happens	Tool/Command
1. Source Checkout	Jenkins pulls latest code from GitHub branch	git clone / git pull
2. Compile	Compiles Java source code	mvn compile
3. Unit Tests	Runs all @Test classes	mvn test
4. Code Quality	Checks code quality rules	SonarQube scan
5. Package	Creates the JAR/WAR file	mvn clean package
6. Build Docker Image	Creates Docker image using Dockerfile	docker build -t ...
7. Push to Registry	Uploads image to AWS ECR	docker push ecr-url/app:tag
8. Deploy to K8s	Updates Kubernetes deployment with new image	kubectl set image / apply
9. Notify	Sends Slack/email on success or failure	Jenkins plugin

3.2 Jenkinsfile — The Pipeline Definition

The Jenkinsfile is the most important Jenkins configuration file. It lives in your project root (alongside pom.xml and Dockerfile). It defines all the pipeline stages in code.

File location: /project-root/Jenkinsfile

```
pipeline {
    agent any // Run on any available Jenkins agent (VM)

    environment {
        // Variables available to all stages
        ECR_REGISTRY = '123456789.dkr.ecr.ap-south-1.amazonaws.com'
        IMAGE_NAME   = 'teammate-reporting-service'
        IMAGE_TAG    = "${BUILD_NUMBER}" // Auto-increments with each build
        KUBE_CONFIG  = credentials('kube-config-secret') // From Jenkins secrets
    }

    stages {
        stage('Checkout') {
```

```

    steps { git branch: 'dev', url: 'https://github.com/org/teammate.git' }
}

stage('Build & Test') {
    steps { sh 'mvn clean package -DskipTests=false' }
}

stage('Code Quality') {
    steps { sh 'mvn sonar:sonar -Dsonar.host.url=$SONAR_URL' }
}

stage('Docker Build') {
    steps {
        sh 'docker build -t $ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG .'
    }
}

stage('Push to ECR') {
    steps {
        sh 'aws ecr get-login-password | docker login --username AWS ...'
        sh 'docker push $ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG'
    }
}

stage('Deploy to K8s') {
    steps {
        sh 'kubectl set image deployment/reporting
$IMAGE_NAME=$ECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG'
    }
}

post {
    success { echo 'Deployment successful!' }
    failure { echo 'Pipeline failed - check logs' }
}
}

```

3.3 Key Jenkinsfile Sections Explained

Section	Purpose	What You Need to Change
agent any	Run on any available Jenkins executor	If you have dedicated agents, specify label
environment {}	Declare env variables for the pipeline	ECR_REGISTRY, IMAGE_NAME per service

Section	Purpose	What You Need to Change
stages {}	The ordered list of pipeline steps	Add/remove stages as needed
sh '...'	Run a shell command on the Jenkins VM	The actual build/docker/kubectl commands
credentials('id')	Securely fetch secrets stored in Jenkins	Must match the credential ID in Jenkins UI
post {}	Actions after pipeline completes (always/success/failure)	Add Slack/email notification
BUILD_NUMBER	Auto-incremented build counter (e.g., 47)	Use as Docker image tag for traceability
when { branch 'main' }	Conditional — run stage only for specific branch	Gate prod deployments to main branch only

3.4 Jenkins Job Configuration (UI — What DevOps Sets Up)

Besides the Jenkinsfile in code, DevOps configures Jenkins itself via the web UI. You should understand this:

Global Tool Configuration (Manage Jenkins)

- **Maven:** Jenkins downloads and manages Maven. Set the Maven version to match your project's pom.xml.
- **JDK:** Configure Java 17 (or whichever version Teammate uses).
- **Docker:** Path to docker binary on the Jenkins VM.
- **kubectl:** Path to kubectl binary for Kubernetes deployments.

Plugins You Must Know

Plugin	Purpose
Git Plugin	Allows Jenkins to clone GitHub repos
Maven Integration	Run Maven goals inside Jenkins jobs
Pipeline	Enables Jenkinsfile-based declarative pipelines
Docker Pipeline	Build/push Docker images from Jenkinsfile
Deploy to Container	Deploy WAR files to Tomcat (used in simpler setups)
AWS Credentials	Store and use AWS access keys securely
Kubernetes CLI	Run kubectl commands from Jenkins
SonarQube Scanner	Code quality analysis integration
Slack Notification	Send build status to Slack channel

3.5 Credentials — Storing Secrets in Jenkins

Jenkins must authenticate with GitHub, AWS ECR, and Kubernetes. These secrets are NEVER hardcoded in Jenkinsfile. They are stored in Jenkins Credential Manager.

Credential ID (example)	Type	Used For
github-creds	Username/Password or SSH Key	Jenkins pulling code from private GitHub repo
aws-ecr-creds	AWS Access Key ID + Secret	docker login to ECR, pushing images
kube-config-secret	Secret File (.kubeconfig)	kubectl connecting to EKS cluster
sonar-token	Secret Text	SonarQube analysis authentication
db-password-dev	Secret Text	DB password injected into dev deployments

```
// How to use a credential in Jenkinsfile
environment {
    AWS_CREDENTIALS = credentials('aws-ecr-creds')
    // Creates: AWS_CREDENTIALS_USR and AWS_CREDENTIALS_PSW automatically
}
```

3.6 Branching Strategy + Environments

Your project uses 4 branches mapped to 4 environments. Jenkins uses branch-specific pipeline behavior:

Branch	Environment	Who Uses It	Jenkins Behavior
dev	DEV Env	Developers	Auto-trigger on every push. Deploy to DEV k8s namespace. Fast feedback loop.
main	QA Env	QA Testers	Trigger on merge from dev. Run full test suite. Deploy to QA namespace after tests pass.
release	UAT / Pilot	Client / UAT Team	Manual approval gate before deploy. Promote same image from QA. Tag image as release candidate.
prod tag	PROD Env	End Users	Manual approval required. SAME image promoted (no rebuild). Zero-downtime rolling deploy.

 **Note:** One image is built in DEV and promoted all the way to PROD. No rebuilding. This ensures what you tested in QA is exactly what runs in PROD.

 **Interview Tip:** 'We follow a GitFlow-inspired branching strategy. The dev branch auto-deploys to DEV environment. PRs to main trigger QA deployment. The release branch requires a manual approval gate before

deploying to UAT and PROD, ensuring the same Docker image artifact flows through all environments without rebuilding.'

Chapter 4 — Kubernetes: Managing Your Containers at Scale

Once Docker packages your app, Kubernetes (K8s) manages running it reliably. Kubernetes ensures your app is always running, handles scaling, and does zero-downtime deployments.

4.1 Why Not Just Use Docker Directly?

Problem with Just Docker	How Kubernetes Solves It
If container crashes, it stays crashed	K8s automatically restarts crashed containers (self-healing)
Manual scaling — you restart more containers	K8s auto-scales based on CPU/memory (HPA)
No built-in load balancing	K8s Service object load-balances across pods
Rolling updates require manual steps	K8s does zero-downtime rolling deployments
Hard to manage 10+ services	K8s manages all services with YAML declarations
No secret/config management	K8s has ConfigMap (config) and Secret (passwords) objects

4.2 Kubernetes Architecture

The Cluster

A K8s cluster = Master Node (Control Plane) + Worker Nodes. In AWS, you use EKS — Amazon manages the master node for you.

Component	Where It Lives	What It Does — Developer Perspective
API Server	Master Node	Entry point for all K8s commands (kubectl talks to this)
etcd	Master Node	Database storing cluster state (what should be running)
Scheduler	Master Node	Decides which worker node a new Pod goes on
Controller Manager	Master Node	Ensures actual state = desired state (restarts failed pods)
kubelet	Each Worker Node	Agent that manages containers on that worker node
kube-proxy	Each Worker Node	Handles network routing between pods
Pod	Worker Node	Smallest deployable unit — contains your container(s)
Docker Engine	Each Worker Node	Actually runs the containers inside pods

4.3 The Key Kubernetes Objects You Work With

Pod

A Pod is the smallest unit in K8s. It wraps one (or rarely more) containers. You never create Pods directly — you create Deployments which manage Pods.

- **Each Pod gets its own IP address** within the cluster network.
- **Pods are ephemeral.** If a Pod dies, a new one starts with a NEW IP. That's why you use Services.

Deployment

A Deployment is a YAML declaration saying: 'I want 3 replicas of the reporting-service image running at all times.' K8s makes it happen.

deployment.yaml — Key Fields

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reporting-service
  namespace: teammate-dev      # Environment separation
spec:
  replicas: 3                  # Run 3 instances (pods)
  selector:
    matchLabels:
      app: reporting-service
  template:
    metadata:
      labels:
        app: reporting-service
    spec:
      containers:
        - name: reporting-service
          image: 123456789.dkr.ecr.ap-south-1.amazonaws.com/reporting:47
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_DATASOURCE_URL
              valueFrom:
                configMapKeyRef:
                  name: teammate-config
                  key: db.url
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: teammate-secrets
                  key: db-password
      resources:
        requests: { cpu: '250m', memory: '512Mi' }
        limits:   { cpu: '500m', memory: '1Gi' }
```

Important Deployment Fields

Field	What It Controls	When You Change It
replicas	Number of pod copies running	Scale up for more traffic, down to save cost
image	Which Docker image version to run	Jenkins updates this on each deploy
namespace	Logical separation per environment	teammate-dev, teammate-qa, teammate-prod
env / configMapKeyRef	Inject config from ConfigMap	DB URL, S3 bucket, API keys per env
secretKeyRef	Inject passwords from Secret	DB passwords, API tokens
resources.requests	Minimum CPU/RAM guaranteed	Tune based on profiling
resources.limits	Maximum CPU/RAM allowed	Prevent one service starving others
containerPort	Port the app inside container listens on	Must match server.port in Spring Boot

Service

A Service gives a stable endpoint (DNS name + stable IP) to access Pods, even as Pods restart and get new IPs.

```

apiVersion: v1
kind: Service
metadata:
  name: reporting-service
  namespace: teammate-dev
spec:
  selector:
    app: reporting-service # Routes to pods with this label
  ports:
    - port: 80             # Port other services call
      targetPort: 8080     # Port the container runs on
  type: ClusterIP         # Internal only (LoadBalancer for external)

```

Service Type	Accessible From	Use Case in Teammate
ClusterIP (default)	Inside the K8s cluster only	Internal microservice communication
NodePort	From outside using Node IP + Port	Testing / non-production
LoadBalancer	From internet via AWS Load Balancer	API Gateway service exposed to frontend/clients

ConfigMap and Secret

These Kubernetes objects store configuration separately from your Docker image. This is how the SAME image runs differently in DEV vs PROD.

```
# ConfigMap - non-sensitive config
kubectl create configmap teammate-config \
  --from-literal=db.url=jdbc:mysql://dev-db:3306/teammate \
  --from-literal=s3.bucket=teammate-audit-dev \
  -n teammate-dev

# Secret - sensitive values (base64-encoded)
kubectl create secret generic teammate-secrets \
  --from-literal=db-password=MySecretPass123 \
  -n teammate-dev
```

Namespace

Namespaces are like virtual clusters within your K8s cluster. Use them to isolate environments:

```
kubectl create namespace teammate-dev
kubectl create namespace teammate-qa
kubectl create namespace teammate-uat
kubectl create namespace teammate-prod
```

 **Note:** All environment differences (DEV vs PROD) come from different namespaces with different ConfigMaps/Secrets. The Docker image is the same.

4.4 Essential kubectl Commands

Command	What It Does
<code>kubectl get pods -n teammate-dev</code>	List all running pods in DEV namespace
<code>kubectl get pods -n teammate-prod</code>	List all running pods in PROD namespace
<code>kubectl describe pod <pod-name> -n teammate-dev</code>	Full details of a pod (use for debugging crashes)
<code>kubectl logs <pod-name> -n teammate-dev</code>	View pod logs (your Spring Boot System.out)
<code>kubectl logs <pod-name> -n teammate-dev -f</code>	Follow/stream live logs (like tail -f)
<code>kubectl logs <pod-name> -n teammate-dev --previous</code>	Logs of previous crashed container
<code>kubectl get deployments -n teammate-dev</code>	List all deployments
<code>kubectl get services -n teammate-dev</code>	List all services (and their IPs/ports)
<code>kubectl apply -f deployment.yaml</code>	Create or update deployment from YAML file
<code>kubectl set image deployment/reporting reporting-service=ecr/image:v2 -n dev</code>	Update image version (what Jenkins does)
<code>kubectl rollout status deployment/reporting -n teammate-dev</code>	Watch rolling update progress

Command	What It Does
kubectl rollout undo deployment/reporting -n teammate-dev	ROLLBACK to previous version
kubectl scale deployment/reporting --replicas=5 -n teammate-dev	Manually scale to 5 pods
kubectl exec -it <pod-name> -n teammate-dev -- bash	Open shell inside a running pod
kubectl get configmap -n teammate-dev	List all configmaps
kubectl delete pod <pod-name> -n teammate-dev	Force restart a pod (new one auto-starts)

4.5 How Zero-Downtime Deployments Work

When Jenkins deploys a new version, Kubernetes does a rolling update:

- Start new pods with the new image (e.g., v48)
- Wait for new pods to pass health checks (readiness probe)
- Only then start terminating old pods (e.g., v47)
- Result: users always hit a healthy pod — zero downtime

If the new image crashes on startup, K8s stops the rollout — old pods keep running. This is self-healing.

 **Interview Tip:** 'In Kubernetes, we use rolling update strategy. When Jenkins deploys build #47, K8s starts new pods with that image, waits for them to become ready (via readiness probes), and only then terminates old pods. If the new version fails health checks, the rollout stops and old pods continue serving traffic. We can also kubectl rollout undo to instantly revert.'

4.6 Health Checks — Liveness and Readiness Probes

These are endpoints in your Spring Boot app that K8s calls to know if the container is healthy:

- **Readiness Probe:** 'Is this pod ready to receive traffic?' If fails, pod is removed from load balancer.
- **Liveness Probe:** 'Is this pod still alive?' If fails, pod is restarted.

```
# In Spring Boot — add actuator dependency
# application.properties
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always
```

```
# In Kubernetes Deployment YAML
livenessProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
```

```
path: /actuator/health
port: 8080
initialDelaySeconds: 20
periodSeconds: 5
```

4.7 EKS — Managed Kubernetes on AWS

Your project uses AWS EKS so you do not manage the K8s master node. DevOps sets up EKS using eksctl:

```
eksctl create cluster --name teammate-cluster \
  --region ap-south-1 \
  --node-type t2.medium \
  --nodes-min 2 --nodes-max 5 \
  --zones ap-south-1a,ap-south-1b
```

Worker nodes are EC2 instances. EKS auto-scales worker nodes based on load (Cluster Autoscaler). Each worker node runs Docker Engine and kubelet.

Chapter 5 — End-to-End Flow: Code to Production

Let us trace exactly what happens from the moment you push a feature to when an auditor sees it in the Teammate app.

5.1 Scenario: You Develop a New API in the Reporting Microservice

Step	What You Do / What Happens	Where
1	Write code in reporting-service module. Test locally with <code>java -jar</code> .	Your laptop / IDE
2	<code>git checkout dev && git pull (get latest)</code>	Local Git
3	<code>git add . && git commit -m 'feat: add audit summary export API'</code>	Local Git
4	<code>git push origin dev</code>	GitHub
5	GitHub webhook triggers Jenkins pipeline for 'dev' branch	Jenkins VM (EC2)
6	Jenkins: <code>git pull</code> → <code>mvn clean package</code> → <code>mvn sonar:sonar</code>	Jenkins VM
7	Jenkins: <code>docker build -t ecr-url/reporting:BUILD_47 .</code>	Jenkins VM (reads Dockerfile)
8	Jenkins: <code>docker push ecr-url/reporting:BUILD_47</code>	AWS ECR
9	Jenkins: <code>kubectl set image deployment/reporting ... -n teammate-dev</code>	EKS cluster
10	K8s rolling update: new pods with BUILD_47 start, old pods stop	EKS Worker Nodes
11	QA team tests the feature at <code>dev.teammate.com</code>	Browser / Postman
12	PR created: <code>dev</code> → <code>main</code> . After merge, same pipeline deploys to QA namespace	Jenkins → EKS QA NS
13	After QA sign-off, PR to release branch. Manual approval in Jenkins.	Jenkins
14	SAME image (BUILD_47) deployed to UAT namespace. Client tests.	EKS UAT NS
15	Production approval. <code>kubectl apply</code> with BUILD_47 to prod namespace.	EKS PROD NS
16	Auditors access <code>https://teammate.com</code> and use the new export API.	Internet → LB → EKS PROD

 **Note:** Notice: the Docker image is built ONCE in Step 7 and promoted through all environments. No rebuilding. This guarantees DEV = QA = UAT = PROD in terms of the binary.

5.2 Environment-Specific Config Summary

Config Item	DEV	QA	PROD
K8s Namespace	teammate-dev	teammate-qa	teammate-prod
DB URL (ConfigMap)	dev-mysql:3306/teammate_dev	qa-mysql:3306/teammate_qa	prod-rds:3306/teammate_prod
S3 Bucket (ConfigMap)	teammate-audit-dev	teammate-audit-qa	teammate-audit-prod
Replicas	1-2	2-3	3-5
Docker Image	ecr/reporting:BUILD_47	ecr/reporting:BUILD_47 (same)	ecr/reporting:BUILD_47 (same)
Log Level	DEBUG	INFO	WARN

5.3 What Changes Per Environment (And Where)

- **application.properties in Spring Boot:** Contains placeholder variables like `${DB_URL}` and `${S3_BUCKET}`. Actual values come from K8s ConfigMap.
- **ConfigMap in K8s:** Has the actual DB URL, S3 bucket name, Redis URL per namespace. DevOps creates these.
- **Secret in K8s:** Has DB password, API tokens. DevOps creates these.
- **Jenkinsfile:** Has when { branch 'release' } gate for manual approval on UAT/PROD.
- **Replicas in deployment.yaml:** Scaled up in PROD for more traffic.

Chapter 6 — Troubleshooting: What to Do When Things Break

As a developer, you need to diagnose issues across the pipeline. Here is the playbook:

6.1 Problem: Jenkins Pipeline Fails

Stage: Build & Test

- **Symptom:** mvn clean package fails.
- **Check:** Jenkins console output for compilation errors, test failures.
- **Fix:** Fix the code. If tests are flaky, you can temporarily use `-DskipTests` (ask team first).

Stage: Docker Build

- **Symptom:** docker build fails.
- **Check:** Is `target/yourapp.jar` present? Did Maven package succeed? Is the JAR name in Dockerfile correct?

```
# Verify JAR exists after mvn package
ls target/*.jar
# Make sure Dockerfile COPY path matches
COPY target/reporting-service-1.0.jar app.jar # MUST match exact JAR name
```

Stage: Push to ECR

- **Symptom:** docker push fails with 'no basic auth credentials'.
- **Check:** Jenkins needs to re-authenticate to ECR (ECR auth tokens expire every 12 hours).
- **Fix:** Ensure the Jenkinsfile includes `aws ecr get-login-password` step before docker push.

Stage: Deploy to K8s

- **Symptom:** `kubectl` command fails with 'Unauthorized'.
- **Check:** The `kubeconfig` credential in Jenkins may be expired or incorrect.
- **Fix:** DevOps needs to refresh the `kubeconfig` credential in Jenkins.

6.2 Problem: Pod is CrashLoopBackOff

This means the container starts, crashes, K8s tries to restart it, it crashes again — looping.

```
# Step 1: Get the pod name
kubectl get pods -n teammate-dev

# Step 2: Describe to see events (why it's crashing)
kubectl describe pod <pod-name> -n teammate-dev

# Step 3: View logs of the crashed container
kubectl logs <pod-name> -n teammate-dev

# Step 4: View logs of the PREVIOUS crash (if current is already restarted)
kubectl logs <pod-name> -n teammate-dev --previous
```

Common causes:

- **Missing env variable:** Spring Boot fails because DB URL not injected. Check ConfigMap and Secret.
- **Wrong port:** App starts on 8081 but K8s readiness probe hits /actuator/health on 8080.
- **OOM Killed:** Container exceeds memory limit. Increase resources.limits.memory in deployment.yaml.
- **DB not reachable:** DB URL incorrect in ConfigMap, or RDS Security Group not allowing K8s pods.

6.3 Problem: App is Running but Returns 5xx

```
# Check live logs
kubectl logs <pod-name> -n teammate-prod -f

# If multiple pods, check all of them
kubectl logs -l app=reporting-service -n teammate-prod

# For Splunk logs (your project uses Splunk for centralized logging)
# Search: index=teammate-prod service=reporting-service level=ERROR
```

6.4 Problem: Need to Rollback Quickly

```
# Instant rollback to previous deployment
kubectl rollout undo deployment/reporting-service -n teammate-prod

# Rollback to a specific revision
kubectl rollout history deployment/reporting-service -n teammate-prod
kubectl rollout undo deployment/reporting-service --to-revision=45 -n teammate-prod
```

 **Note:** Rollback in K8s is instant — it switches to the previous image. No need to rebuild or redeploy. This is why Docker image tagging per build number is critical.

Chapter 7 — Interview Q&A for Your Project

Use these answers tailored to the Teammate audit application:

7.1 Docker Questions

Q: What is Docker and why do you use it?

Docker is a containerization platform that packages our application code along with all its dependencies — Java runtime, libraries, configuration — into a portable image. In Teammate, we containerize each microservice (reporting, document management, notifications, background jobs) separately. This ensures the reporting service that passes QA testing is byte-for-byte identical to what runs in production, eliminating environment-specific bugs.

Q: What is in your Dockerfile?

Our Dockerfile uses `openjdk:17-jdk-slim` as the base image (matching our project's Java version), sets `/app` as the working directory, copies the Maven-built JAR into the container, exposes port 8080, and uses `ENTRYPOINT [java, -jar, app.jar]` to start the application. We do not bake environment-specific configuration into the image — database URLs, S3 bucket names, and credentials are injected at runtime via Kubernetes ConfigMaps and Secrets.

Q: What is Docker Compose? Do you use it?

Docker Compose is a tool for running multi-container applications locally. We use it for local development setups where a developer needs to run the service alongside its dependencies (MySQL, Redis) in one command using `docker-compose up`. However in production, Kubernetes manages multi-container orchestration. We do not rely on Docker Compose for any environment above local development.

7.2 Jenkins / CI-CD Questions

Q: Explain your CI/CD pipeline.

Our Jenkins pipeline has these stages: Source Checkout from GitHub, Maven Build and Unit Tests, SonarQube code quality scan, Docker image build, push to AWS ECR, and `kubectl` deployment to Kubernetes. The pipeline is defined as a Jenkinsfile checked into the repository alongside the code, so pipeline changes are versioned and reviewed like any other code change.

Q: How do you handle multiple environments in Jenkins?

We use branch-based pipeline behavior. The dev branch auto-deploys to the DEV Kubernetes namespace on every push. Merging to main triggers QA deployment. The release branch requires a manual approval step in Jenkins before deploying to UAT. Production deployments require a second manual approval and are auditable. Crucially, we build the Docker image only once on the dev branch and promote that exact image artifact across all environments — we never rebuild for QA or PROD.

Q: How does Jenkins authenticate with AWS?

Jenkins uses AWS IAM credentials stored in Jenkins Credential Manager (not hardcoded in the Jenkinsfile). For ECR, it runs `aws ecr get-login-password` to obtain a temporary auth token before docker push. For Kubernetes, it uses a kubeconfig file stored as a Jenkins secret that grants it access to the EKS cluster. The IAM role attached to the Jenkins EC2 instance also has ECR push and EKS describe permissions.

7.3 Kubernetes Questions

Q: What Kubernetes objects do you work with?

We primarily use Deployments to manage our microservice pods with replica counts and rolling update strategy, Services to expose pods with stable endpoints (ClusterIP for internal communication, LoadBalancer for the API Gateway), ConfigMaps to store environment-specific non-sensitive configuration like DB URLs and S3 bucket names, Secrets for sensitive data like DB passwords, Namespaces for environment isolation (teammate-dev, teammate-qa, teammate-prod), and HorizontalPodAutoscaler for auto-scaling based on CPU.

Q: How does zero-downtime deployment work in Kubernetes?

Kubernetes uses a rolling update strategy by default. When Jenkins deploys a new image, K8s starts new pods with the new version and waits for them to pass the readiness probe (we expose /actuator/health via Spring Boot Actuator). Only after new pods are ready does K8s terminate old pods. If the new pods fail to start or fail health checks, the rollout automatically stops and old pods continue serving traffic. We can also `kubectl rollout undo` to instantly revert to the previous version.

Q: How do your microservices communicate in Kubernetes?

Internal service-to-service communication uses Kubernetes DNS. Each Service gets a DNS name like `reporting-service.teammate-dev.svc.cluster.local`. Spring Boot RestTemplate or WebClient uses these internal DNS names. Synchronous calls use RestTemplate, async event-driven communication uses Kafka. Services are not exposed externally unless they need to be — internal services use ClusterIP type.

Q: How do you handle configuration differences between environments?

We use Kubernetes ConfigMaps for environment-specific non-sensitive config and Secrets for passwords. Our Spring Boot application.properties uses placeholder variables like `${DB_URL}` and `${S3_BUCKET}`. The actual values are injected by Kubernetes from the ConfigMap in the relevant namespace. DevOps maintains separate ConfigMaps per namespace — teammate-dev has dev DB endpoints, teammate-prod has production RDS endpoints. The Docker image never changes — only the injected config differs.

7.4 Architecture Questions

Q: Describe your application architecture.

Teammate is an internal auditing application currently in a hybrid architecture — transitioning from a monolith to microservices. The monolith handles core transactional audit workflows. We have extracted reporting, document management, notifications, and background jobs into independent microservices because they are either resource-intensive, event-driven, or need independent scaling. All services run in Docker containers managed by Kubernetes on AWS EKS. The monolith serves as the API Gateway for initial request routing, backed by a shared MySQL database (shared during migration phase). We use Redis for caching, Kafka for async messaging, Splunk for centralized logging, and Datadog for monitoring.

Q: Which services did you extract to microservices first and why?

We extracted reporting, document management, notification, and background job services first. These were chosen because: Reporting is resource-intensive — generating audit reports under load affects the monolith. Document management needed S3 integration isolated independently. Notifications are event-driven and better served by Kafka consumers in a separate service. Background jobs needed independent scaling and scheduling without blocking the monolith thread pool. Core audit workflow logic stayed in the monolith to maintain transactional consistency during the migration phase.

Chapter 8 — Quick Reference Command Cards

8.1 Docker Quick Reference

Task	Command
Build image	<code>docker build -t myapp:1.0 .</code>
Run container	<code>docker run -d -p 8080:8080 myapp:1.0</code>
View logs	<code>docker logs -f <container-id></code>
Shell into container	<code>docker exec -it <container-id> bash</code>
List images	<code>docker images</code>
List containers	<code>docker ps -a</code>
Stop container	<code>docker stop <container-id></code>
Remove image	<code>docker rmi <image-id></code>
Login to ECR	<code>aws ecr get-login-password docker login --username AWS --password-stdin <ecr-url></code>
Push to ECR	<code>docker push <ecr-url>/myapp:1.0</code>

8.2 kubectl Quick Reference

Task	Command
List pods	<code>kubectl get pods -n teammate-dev</code>
Pod details	<code>kubectl describe pod <name> -n teammate-dev</code>
View logs	<code>kubectl logs <name> -n teammate-dev -f</code>
Previous crash logs	<code>kubectl logs <name> -n teammate-dev --previous</code>
List services	<code>kubectl get svc -n teammate-dev</code>
Apply YAML	<code>kubectl apply -f deployment.yaml</code>
Update image	<code>kubectl set image deployment/svc svc=ecr/img:v2 -n ns</code>
Rollback	<code>kubectl rollout undo deployment/<name> -n teammate-dev</code>
Scale pods	<code>kubectl scale deployment/<name> --replicas=4 -n teammate-dev</code>
Shell into pod	<code>kubectl exec -it <name> -n teammate-dev -- bash</code>
Get configmaps	<code>kubectl get configmap -n teammate-dev</code>
Delete pod (restart)	<code>kubectl delete pod <name> -n teammate-dev</code>

Task	Command
Watch rollout	kubectl rollout status deployment/<name> -n teammate-dev

8.3 Linux Quick Reference (on EC2 / Jenkins VM)

Task	Command
Check running Java	ps -ef grep java
Check port in use	netstat -tlnp grep 8080
Check disk space	df -h
Check memory	free -h
View file tail	tail -f /var/log/app.log
Check Jenkins status	sudo systemctl status jenkins
Install software	sudo yum install git OR sudo apt install git
Maven package	mvn clean package -DskipTests
View env variables	env grep SPRING

8.4 Git Branching Quick Reference

Action	Command	Notes
Start feature	git checkout dev && git pull	Always pull latest dev first
Create feature branch	git checkout -b feature/audit-export	From dev branch
Push feature	git push origin feature/audit-export	
PR: feature → dev	Via GitHub UI	Code review required
PR: dev → main	Via GitHub UI	Triggers QA deploy
PR: main → release	Via GitHub UI	Triggers UAT + prod gate
Hotfix	Branch from release, fix, PR back to release + main	Do NOT branch from dev
Check branch	git branch -a	List all branches
Sync with upstream	git pull origin dev	Before starting any work

End of Document

Teammate Audit Application — Developer DevOps Guide